

# From Cars to Clinics

Service-Oriented Middleware for Networked Medical Devices

---

Fabian Franzelin

2026-09-24



## Cold Open

---

## A patient at home. A clinician in the hospital.

- A wearable sensor on the belt streams 3D acceleration at 100 Hz.
- A clinician's dashboard shows the patient's gait cadence in real time.
- Multiple sclerosis, follow-up, no clinic visit needed today.

## One question drives the next 20 minutes

*How does the dashboard actually talk to the sensor — safely, interoperably, in real time?*

# Framing

---

## Prerequisites I assume

From your previous semesters:

| Semester | Course                         | What I will reuse                |
|----------|--------------------------------|----------------------------------|
| 3        | Kommunikationsnetze            | OSI, TCP/UDP, HTTP, sockets      |
| 3        | Multimodale Signalverarbeitung | Sampling, sensor data streams    |
| 4        | Verteilte Systeme              | SOA, multi-tier, security        |
| 4        | Softwaretechnik                | IEC 62304 software lifecycle     |
| 4        | Standards und Prozesse         | Medical workflow vs. informatics |

## Where we are going

1. Requirements of a networked medical device
2. **Deep dive:** SDC / IEEE 11073
3. Automotive lens — what transfers
4. Where this fits in the wider e-health stack
5. Research directions I would bring to Reutlingen

# Requirements

---

## What a networked medical device must offer

1. **Interoperability** — multi-vendor plug-and-play in the OR / at home
2. **Real time** — alarms, closed-loop control, streaming vitals
3. **Safety** — IEC 62304, ISO 14971, IEC 60601-1
4. **Security** — IEC 62443, MDR, threat modelling
5. **Traceability** — who did what, when, from which device
6. **Scalability** — device → room → hospital → network

## A single scenario forces all six

- Wearable sends *TidalVolume*, *HeartRate*, *StepCadence*.
- Dashboard subscribes, renders, raises an alert if a threshold is crossed.
- Multiple vendors. Multiple networks. Multiple regulatory jurisdictions.

*These are the non-functional requirements. They drive the architecture.*

## Deep dive — SDC / IEEE 11073

---

## IEEE 11073 — a family, not a single standard

| Part            | Scope                                         |
|-----------------|-----------------------------------------------|
| 11073-10207     | Domain Information & Service Model (BICEPS)   |
| 11073-20701     | Service-Oriented Device Connectivity (SDC)    |
| 11073-20702     | Medical DPWS binding (transport)              |
| 11073-10101(nu) | Nomenclature codes for vitals, alarms, events |

**BICEPS** = Basic Integrated Clinical Environment Protocol Specification.

## The three roles in SDC

- **Provider** — a medical device that offers services (a ventilator).
- **Consumer** — a system that consumes them (a monitor, a dashboard).
- **SDC network** — discoverable, self-describing, secured.

Everything above the network layer is **domain-specific medical semantics**.

# BICEPS in one picture

Every provider hosts an **MDIB** (Medical Device Information Base):

MDIB

Descriptors (what exists: static structure)

MDS (Medical Device System)

VMD (Virtual Medical Device)

Channel

Metric ← e.g., StepCadence

States (current values: dynamic content)

NumericMetricState (single scalar)

StringMetricState

RealTimeSampleArrayState (waveform)

AlertConditionState

AlertSignalState

## The four service groups

- **Get Service** — pull the MDIB, one snapshot.
- **Set Service** — change device settings (with safety context!).
- **Description / State Event Service** — push updates to subscribers.
- **Context Service** — who is the patient, which location, which workflow?

## Safety Contexts — the hard part

A *Set* operation is not just an RPC. The consumer must include a **Safety Context** proving:

- the right **patient** is targeted (PatientContext),
- the right **location** (LocationContext),
- the right **workflow** (WorkflowContext),
- and that the operation is **permitted right now** (Participant Keys).

*This is where SDC earns the word "medical".*

- **WS-Discovery** over UDP multicast — how devices find each other
- **SOAP/HTTP(S)** over TCP — for messages
- **WS-Eventing** — for subscriptions
- **TLS** — transport security
- **WS-Security + XML Signature** — message integrity above TLS

Layering, cleanly separated.

## Sequence: discovery → subscribe → stream

| Consumer                          | Provider (wearable)       |
|-----------------------------------|---------------------------|
|                                   |                           |
| --- Probe (WS-Discovery) ----->   |                           |
| <-- ProbeMatch -----              |                           |
|                                   |                           |
| --- GetMdib ----->                |                           |
| <-- MDIB (descriptors + states)-- |                           |
|                                   |                           |
| --- Subscribe (State Events) ---> |                           |
| <-- SubscribeResponse -----       |                           |
|                                   |                           |
| <== EpisodicMetricReport =====    | (StepCadence = 108/min)   |
| <== EpisodicMetricReport =====    | (StepCadence = 106/min)   |
| <== EpisodicAlertReport =====     | (Cadence below threshold) |

## Look at the corner of your screen

The dashboard from the opening slide has been running through this whole section.

Provider streams synthetic 3D-accelerometer data. Consumer computes rolling cadence. No cloud. No configuration. Localhost only.

*We will come back to this at the end.*

## Automotive lens

---

## Same problem, different domain

In eight years at Bosch I built middleware for a car with 100+ ECUs. The requirements list I showed you earlier?

*Word for word the same, minus one column.*

| Concern                | Automotive               | Medical                                   |
|------------------------|--------------------------|-------------------------------------------|
| Service-oriented comms | SOME/IP (AUTOSAR AP)     | SDC (IEEE 11073-20701)                    |
| Real-time streaming    | DDS, iceoryx (zero-copy) | DDS, SDC waveform states                  |
| Safety lifecycle       | ISO 26262 (ASIL A-D)     | IEC 62304 (Class A/B/C) + ISO 14959       |
| Cybersecurity          | ISO/SAE 21434            | IEC 62443 + MDR                           |
| Process maturity       | Automotive SPICE         | ISO 13485 + IEC 62304                     |
| Risk acceptance        | Residual risk only       | Residual risk <b>vs. clinical benefit</b> |

# What transfers, what does not

## Transfers well:

- Service-oriented architectural patterns
- QoS-based real-time streaming
- Risk-based V-model and traceability discipline
- Test infrastructure and simulation (my digital-twin work at Bosch)

## Does *not* transfer:

- **Domain semantics** — BICEPS is medicine, VLANs are cars
- **Clinical benefit** in the risk equation — unique to medicine
- **Patient / workflow context** — no analogue in the vehicle

*Architectural patterns are transferable. Domain semantics are not.*

That is where the real engineering happens. And that is what I want to teach and research at Reutlingen.

## Where SDC fits

---

## One wearable, one dashboard — but there is more

```
+-----+
| Physician office |
| / EHR / national |
| telematics platform |
+-----+-----+
| FHIR / TI-Messenger / KIM
| (system-to-system)
+-----+-----+
| Hospital gateway |
+-----+-----+
| SDC / IEEE 11073
| (device-to-device)
+-----+-----+
| Bedside monitor, |
```

## Two layers of the same stack

- Above the device layer: the **Telematikinfrastuktur** — topic of my German talk earlier today.
- Below the device layer: *this* talk.

*A "networked medical system" is not one protocol. It is a consistent architectural stance across many.*

## Research at Reutlingen

---

# What I would bring to CaMed

Three concrete directions — each with an existing anchor at Reutlingen:

1. **SDC as the substrate for the connected OR** — image-guided procedures, real-time device orchestration, 5G-connected surgical scenarios.
2. **Digital-twin-based V&V of networked device configurations** — carrying over the Bosch/ETH work on automotive digital twins.
3. **Uncertainty quantification for sensor-fusion pipelines** — from wearables to closed-loop clinical decisions (my PhD, applied to medicine).

- Project-based, end-to-end: requirements → architecture → code → V&V → regulatory dossier.
- Everything as code, everything reproducible, everything reviewable.
- This lecture is the smallest example: 20 minutes of talk, one git repository.

## Closing the circle

---

## The dashboard you saw at minute one

- Was running from this repository.
- Provider + consumer + dashboard in ~200 lines of Python.
- Synthetic gait data — deterministic, reproducible, CC0.

**These slides** were written in Org, rendered as reveal.js, versioned in Git. The Beamer PDF was produced by the same source, via Pandoc, in CI.

*These slides. This demo. Reproducible. Public domain.*

Any of you can run make demo tonight.

Thank you — questions?

## Backup slides

---

## BICEPS metric types in detail

- `NumericMetricState` — single scalar with unit + timestamp
- `StringMetricState` — textual value
- `EnumStringMetricState` — enumerated value
- `RealTimeSampleArrayMetricState` — waveform, sampled data
- `DistributionSampleArrayMetricState` — statistical distributions

- Software development plan
- Software requirements specification
- Software architecture
- Detailed design
- Unit implementation + verification
- Integration + integration testing
- System testing
- Release documentation
- Problem resolution + change control across the lifecycle

## SDC vs. FHIR — one line

- **SDC**: real-time, device-to-device, inside a clinical space, milliseconds.
- **FHIR**: system-to-system, cross-institution, over the internet, seconds.

They are complementary, not competing.